

VitaCare

Rapport de compromis techniques et décisions de conception

Nathéo MILLIET-TREBOUX / Elliott MUNOZ / José AFONSO LOPES

Table des matières

1. Introduction	2
2. Architecture du projet	2
3. Ce qu'on a abandonné ou simplifié	2
React Router	2
Axios remplacé par fetch	3
JWT remplacé par les sessions PHP	3
L'architecture frontend séparée (Vite + dossier frontend)	3
4. Difficultés rencontrées	4
La synchronisation des créneaux en temps réel	4
Le panier multi-étapes	4
La gestion des rôles sans middleware	4
Le débogage sans outil de build	4
5. Solutions proposées qui ne convenaient pas	4
L'architecture REST stricte	4
Redux / Context API pour la gestion d'état	5
Le JWT généré par IA	5
6. Limites actuelles du projet	5
Ce qui ne fonctionne pas parfaitement	5
Ce qui est incomplet ou partiel	5
Ce qui limite la portabilité	5
7. Bilan	6

1. Introduction

VitaCare est une plateforme web qu'on a développée dans le cadre du projet de Web dynamique. Elle permet à des patients de consulter des services médicaux, de faire des réservations, de gérer un panier et de suivre leur historique. Il y a aussi des espaces pour les intervenants (créer des services, gérer des créneaux) et les administrateurs (gérer les comptes utilisateurs).

Ce rapport présente les choix qu'on a faits pendant le développement, ce qu'on a dû abandonner ou simplifier, les difficultés qu'on a rencontrées et les limites de ce qu'on a produit. L'idée n'est pas de prétendre que tout s'est passé comme prévu, mais d'expliquer honnêtement comment le projet a évolué.

2. Architecture du projet

Le site tourne sous WAMP (Apache + MySQL + PHP). On a utilisé PHP natif côté serveur pour gérer les requêtes vers la base de données et les sessions, et on a intégré React directement dans les fichiers .php via Babel in-browser — c'est-à-dire sans Node.js ni outil de build comme Vite.

La base de données MySQL contient 7 tables (users, services, reservations, activites, inscriptions_activites, notifications, programmes) et gère 3 rôles : Patient, Intervenant et Administrateur, avec des redirections automatiques à la connexion selon le rôle.

En termes de pages, on a développé et fait fonctionner : la page d'accueil avec les services dynamiques et le widget des prochains rendez-vous, le catalogue avec filtres et barre de recherche, la page de réservation avec calendrier et créneaux en temps réel, le panier en 3 étapes, le profil utilisateur, les notifications, l'authentification complète, et les deux dashboards (admin et intervenant).

3. Ce qu'on a abandonné ou simplifié

Plusieurs choses étaient prévues au départ et ont finalement été abandonnées ou revues à la baisse. Voici les principales.

React Router

Au départ on voulait utiliser React Router pour naviguer entre les pages sans rechargement, comme une vraie Single Page Application. On a vite compris que ça n'allait pas fonctionner avec notre setup : React Router s'utilise normalement avec un projet Node.js (Vite ou Create React App), pas avec un React intégré dans des fichiers .php via Babel.

On a donc gardé la navigation classique entre fichiers .php. C'est moins fluide (il y a un rechargement à chaque changement de page), mais ça correspondait à ce qu'on savait faire et c'était cohérent avec notre architecture.

On perd en fluidité d'interface, mais on gagne en simplicité et on évite de se battre contre notre propre environnement.

Axios remplacé par fetch

On avait pensé utiliser Axios pour les appels HTTP côté client. En pratique, sans npm disponible dans notre contexte, ça aurait nécessité d'importer Axios via un CDN dans chaque fichier PHP. On a préféré utiliser l'API fetch native du navigateur, qui fait exactement la même chose et ne demande aucune dépendance externe.

- fetch est un peu plus verbeux que Axios pour gérer les erreurs, mais ça ne pose pas de vrai problème pour notre usage.

JWT remplacé par les sessions PHP

L'authentification par tokens JWT avait été évoquée en début de projet. On a assez rapidement décidé de ne pas partir là-dedans : les JWT sont conçus pour des API REST sans état côté serveur, alors qu'on est sur une architecture PHP classique avec des sessions. Mélanger les deux aurait compliqué inutilement le code.

Les sessions PHP native, c'est ce qu'on a vu en cours, ça marche bien pour ce qu'on veut faire, et ça permet de gérer les rôles et les redirections sans surcharge.

- Les sessions PHP sont liées au serveur (pas stateless), mais dans notre contexte c'est largement suffisant et bien plus simple à maintenir.

L'architecture frontend séparée (Vite + dossier frontend)

On avait envisagé de séparer complètement le frontend (React/Vite) du backend (PHP). Cette idée a été abandonnée assez tôt pour plusieurs raisons concrètes :

- WAMP ne gère pas un serveur Node.js en parallèle sans configuration avancée qu'on ne maîtrisait pas.
- Avoir un frontend sur le port 5173 et un backend sur le port 80 aurait créé des problèmes de CORS qu'on aurait mis du temps à résoudre.
- Ça aurait aussi compliqué la gestion des sessions PHP, qui n'auraient pas fonctionné de la même façon avec une API stateless.

L'intégration de React directement dans les fichiers .php via Babel était moins "propre" architecturalement, mais elle nous a permis d'avancer sans blocage.

- Le JSX est transpilé à la volée dans le navigateur à chaque chargement, ce qui est moins performant qu'un bundle compilé. C'est acceptable en développement, mais ça ne serait pas viable en production.

4. Difficultés rencontrées

La synchronisation des créneaux en temps réel

La page de réservation affiche les créneaux disponibles dynamiquement selon l'intervenant et la date choisie. La difficulté principale était de faire en sorte que l'affichage côté client soit toujours cohérent avec ce qu'il y a en base — notamment quand un créneau venait d'être réservé par quelqu'un d'autre.

On a géré ça avec des appels fetch vers des endpoints PHP qui retournent les créneaux disponibles en JSON. Ça fonctionne, mais il n'y a pas de protection contre les soumissions simultanées (si deux personnes réservent le même créneau en même temps, la deuxième requête peut passer avant que la première soit traitée).

Le panier multi-étapes

Le panier fonctionne en 3 étapes (récapitulatif → paiement → confirmation). Gérer l'état entre ces étapes avec React (useState côté client) tout en maintenant la cohérence avec les sessions PHP côté serveur a été assez délicat. Il a fallu bien réfléchir à ce qui est stocké où, et s'assurer qu'un rechargement de page ne cassait pas le parcours.

La gestion des rôles sans middleware

Dans une architecture avec un framework (Laravel, Symfony...), la vérification des droits d'accès se gère généralement dans un middleware centralisé. Ici, on devait répéter la vérification du rôle au début de chaque fichier PHP. C'est fastidieux et potentiellement source d'oublis — on a dû relire plusieurs fois l'ensemble des pages pour s'assurer qu'aucune n'était accessible sans contrôle de rôle.

Le débogage sans outil de build

Sans Vite ou Webpack, les erreurs dans le code JSX sont parfois moins bien rapportées. Babel in-browser transpile à la volée et les messages d'erreur dans la console sont parfois peu explicites. On a perdu du temps sur des erreurs de syntaxe JSX qui auraient été immédiatement signalées avec un bundler.

5. Solutions proposées qui ne convenaient pas

Pendant le développement, on a consulté pas mal de documentation, de tutoriels et utilisé des outils d'IA pour débloquer certaines situations. Plusieurs suggestions se sont révélées inadaptées à notre contexte.

L'architecture REST stricte

La plupart des tutoriels React recommandent une séparation stricte frontend/backend avec une API REST. C'est une bonne pratique, mais dans notre cas ça ne collait pas avec l'environnement WAMP et le niveau du cours. On a adapté en faisant des fichiers PHP qui servent parfois du HTML, parfois du JSON selon le contexte, ce qui est moins propre mais pragmatique.

Redux / Context API pour la gestion d'état

Face aux difficultés de gestion d'état sur le panier, des outils comme Redux ou le Context API de React ont été suggérés (notamment par des outils d'IA). Le problème : dans un contexte React sans npm (Babel in-browser), les installer demande des imports CDN supplémentaires et une configuration pas évidente. On a gardé useState local, ce qui suffit pour notre usage même si c'est moins élégant.

Le JWT généré par IA

À un moment, on a demandé à un outil d'IA de nous aider à implémenter l'authentification. Il nous a fourni du code JWT avec la librairie PHP firebase/php-jwt. Le code était techniquement correct en isolation, mais complètement incohérent avec le reste de notre projet : on avait déjà des sessions PHP en place, et greffer du JWT par-dessus aurait créé une double couche d'authentification sans intérêt. On a rejeté cette suggestion et gardé nos sessions.

On a donc compris que les suggestions automatiques (IA ou doc générique) ne tiennent pas compte du contexte spécifique du projet. Il faut toujours évaluer si la solution proposée s'intègre bien dans ce qui existe déjà.

6. Limites actuelles du projet

Ce qui ne fonctionne pas parfaitement

- Performance côté client : Babel transpile le JSX à chaque chargement de page, ce qui est sensiblement plus lent qu'un bundle pré-compilé sur les pages avec beaucoup de composants.
- Pas de protection contre les réservations simultanées : deux utilisateurs peuvent théoriquement réserver le même créneau en même temps si leurs requêtes arrivent quasi simultanément.
- Le paiement est entièrement simulé : on valide un formulaire de carte bancaire factice, sans aucune intégration réelle.
- Pas de tests automatisés : tout a été vérifié manuellement, ce qui n'est pas idéal pour détecter des régressions.

Ce qui est incomplet ou partiel

- Les notifications n'apparaissent pas en temps réel : elles sont créées en base à chaque réservation, mais l'utilisateur doit recharger la page pour les voir.
- L'adaptation mobile est partielle : le site a été conçu principalement pour desktop, certaines pages ne s'affichent pas bien sur petits écrans.

Ce qui limite la portabilité

- Le projet nécessite WAMP, un import SQL manuel et un placement dans le bon dossier. Il n'y a pas de script d'installation automatisé.
- Il ne serait pas déployable tel quel sur un hébergement classique sans refactoring, notamment à cause de Babel in-browser et de l'absence de build.

7. Bilan

En résumé, on a fait beaucoup de compromis pour pouvoir livrer quelque chose de fonctionnel dans les délais impartis. Certains de ces compromis étaient vraiment nécessaires (abandonner React Router et Vite vu notre environnement), d'autres ont été des simplifications qu'on referait probablement différemment avec plus de temps (la gestion d'état, la protection contre les réservations simultanées).

Ce qui a le mieux marché, c'est d'adapter les solutions à notre contexte réel plutôt que de chercher à tout prix à appliquer les "bonnes pratiques" vues en ligne. L'intégration React via Babel dans les fichiers PHP n'est pas une architecture recommandée dans les tutoriels, mais elle nous a permis de progresser sans bloquer le reste de l'équipe sur de la configuration.

La principale limite qu'on identifie aujourd'hui est le manque de tests et la répétition de la logique de contrôle d'accès dans chaque fichier. Si on devait continuer à maintenir ce projet, ce seraient les deux premiers points à corriger.